

Лабораторная работа №1

Тетеркина Любовь, гр. 4404

Задача 1. Написать программу, где каждый поток печатает свой идентификатор, количество потоков всего и строку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

```
*****
** Visual Studio 2019 Developer PowerShell v16.11.60
** Copyright (c) 2021 Microsoft Corporation
*****
PS C:\Users\teterkina.l.s\source\repos\task1> cd Debug
PS C:\Users\teterkina.l.s\source\repos\task1\Debug> .\task1.exe
Enter number of threads: 8
Thread 0 of 8: Hello World
Thread 2 of 8: Hello World
Thread 1 of 8: Hello World
Thread 3 of 8: Hello World
Thread 6 of 8: Hello World
Thread 5 of 8: Hello World
Thread 4 of 8: Hello World
Thread 7 of 8: Hello World
PS C:\Users\teterkina.l.s\source\repos\task1\Debug> .\task1.exe
Enter number of threads: 8
Thread 0 of 8: Hello World
Thread 1 of 8: Hello World
Thread 3 of 8: Hello World
Thread 2 of 8: Hello World
Thread 4 of 8: Hello World
Thread 5 of 8: Hello World
Thread 6 of 8: Hello World
Thread 7 of 8: Hello World
PS C:\Users\teterkina.l.s\source\repos\task1\Debug> .\task1.exe
Enter number of threads: 8
Thread 0 of 8: Hello World
Thread 1 of 8: Hello World
Thread 5 of 8: Hello World
Thread 3 of 8: Hello World
Thread 2 of 8: Hello World
Thread 6 of 8: Hello World
Thread 4 of 8: Hello World
Thread 7 of 8: Hello World
PS C:\Users\teterkina.l.s\source\repos\task1\Debug>
```

Рис. 1. Скриншот вывода программы task1

Ответ на вопрос: Вывод не идентичен от запуска к запуску: порядок строк меняется, т.к. потоки выполняются параллельно, а порядок их доступа к общему cout определяется планировщиком операционной системы и не детерминирован.

Директива `#pragma omp critical` защищает от одновременной записи в cout (символы не перемешиваются), но не задаёт очерёдность.

Набор строк всегда одинаков: N строк, id от 0 до N-1, каждое число встречается ровно один раз. Меняется только их последовательность.

Задача 2. Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: $b[i] = (a[i-1] + a[i] + a[i+1])/3.0$. Запустите программу с 8-ю процессами при различных типах распределения работ.

```
*****
** Visual Studio 2019 Developer PowerShell v16.11.60
** Copyright (c) 2021 Microsoft Corporation
*****
PS C:\Users\teterkina.1.s\source\repos\task2> cd Debug
PS C:\Users\teterkina.1.s\source\repos\task2\Debug> .\task2.exe
Enter size of array: 16000
Enter number of threads: 8

--- schedule(static) ---
Time: 0.0009889 s

--- schedule(dynamic) ---
Time: 0.0009024 s

--- schedule(guided) ---
Time: 0.0009026 s

--- schedule(static, 64) ---
Time: 0.0009047 s

--- schedule(dynamic, 64) ---
Time: 0.0008919 s

b[1]=1 b[100]=100 b[N-2]=15998
PS C:\Users\teterkina.1.s\source\repos\task2\Debug>
```

Рис. 2. Скриншот вывода программы task2

Анализ результатов

Контрольные значения корректны: $b[1] = 1$, $b[100] = 100$, $b[N-2] = 15998$.

Разница между типами schedule выражена, потому что:

- Первое измерение (static) включает накладные расходы на создание пула потоков OpenMP.
- Варианты с $chunk = 64$ дают лучшую балансировку при малом размере блока.
- На такой маленькой задаче (одна арифметическая операция на итерацию) накладные расходы на синхронизацию сравнимы с полезной работой.

Вывод: на равномерной нагрузке (все итерации одинаковы по стоимости) разница между типами schedule невелика; тип влияет сильнее при неравномерной нагрузке или большом объёме данных.

Задача 3. Модифицируйте задачу 1 так, что бы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения. Постарайтесь найти как можно больше.

```

PS C:\Users\teterkina.1.s\source\repos\task3\Debug> .\task3.exe
Enter number of threads: 8
Enter method (1-5): 1
[Method 1: ordered]
Thread 0 (iter 7)
Thread 1 (iter 6)
Thread 2 (iter 5)
Thread 3 (iter 4)
Thread 4 (iter 3)
Thread 5 (iter 2)
Thread 6 (iter 1)
Thread 7 (iter 0)
PS C:\Users\teterkina.1.s\source\repos\task3\Debug> .\task3.exe
Enter number of threads: 8
Enter method (1-5): 2
[Method 2: critical + reverse counter]
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
PS C:\Users\teterkina.1.s\source\repos\task3\Debug> .\task3.exe
Enter number of threads: 8
Enter method (1-5): 3
[Method 3: barrier + step]
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
PS C:\Users\teterkina.1.s\source\repos\task3\Debug> .\task3.exe
Enter number of threads: 8
Enter method (1-5): 4
[Method 4: master + flags array]
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0
PS C:\Users\teterkina.1.s\source\repos\task3\Debug> .\task3.exe
Enter number of threads: 8
Enter method (1-5): 5
[Method 5: single + reverse loop]
Thread 7
Thread 6
Thread 5
Thread 4
Thread 3
Thread 2
Thread 1
Thread 0

```

Рис. 3. Скриншот вывода программы task3

Описание способов

№	Название	Механизм
1	ordered	Директива ordered гарантирует порядок итераций как в последовательной программе
2	critical + счётчик	Поток ждёт, пока переменная next не станет равна его id; печатает и уменьшает next

№	Название	Механизм
3	barrier + шаг	На шаге s печатает поток с id num-1-s; между шагами – барьер
4	master + флаги	Все потоки выставляют флаг готовности, главный печатает в обратном порядке
5	single + обратный цикл	Один поток печатает все id в обратном порядке через single

Выводы:

- OpenMP позволяет легко создавать многопоточные приложения на C++ с помощью директив компилятора.
- `#pragma omp parallel` создаёт команду потоков; `omp_get_thread_num()` и `omp_get_num_threads()` дают информацию о них.
- Порядок вывода недетерминирован – это фундаментальное свойство параллельных программ.
- `#pragma omp critical` защищает от перемешивания символов, но не задаёт очерёдность.
- `schedule` влияет на распределение работы; для равномерной нагрузки подходит `static`, для неравномерной – `dynamic/guided`.
- Средства синхронизации (`ordered`, `barrier`, `master`, `single`) позволяют принудительно упорядочивать работу потоков.